

JavaScript Interview Preparation Guide

Tailored for 1–2 YOE Frontend & Node.js Developers | 11 Core Modules • 175 Target Questions

Learning Path Strategy: Master topics in sequential progression levels (Level 1 through Level 11). Focus on mechanism comprehension rather than answer memorization.

LEVEL 1
Fundamentals

LEVEL 2
Scope & Hoisting

LEVEL 3
Functions & Closures

LEVEL 4
Objects & Prototypes

LEVEL 5
Arrays & Iteration

LEVEL 6
Async JS & Promises

LEVEL 7
Event Loop Engine

LEVEL 8
DOM & Browser APIs

LEVEL 9
Modern ES6+ Syntax

1. CORE JAVASCRIPT FUNDAMENTALS

~20 Questions

1. Variable Declarations

Compare `var`, `let`, and `const` regarding scope, redeclaration, re-assignment, and hosting behavior.

2. Scope & Lexical Scope

Explain global, function, and block scope. How does the outer environment binding work lexically?

3. Temporal Dead Zone (TDZ)

Define TDZ for `let` and `const`. Why does accessing them before declaration throw a `ReferenceError`?

4. Primitive vs Reference Types

How are primitives stored in memory (stack) versus objects/arrays (heap)? Implications for memory reference.

5. Equality Operators

Explain the difference between abstract equality (`==`) and strict equality (`===`) with implicit type conversion rules.

6. Type Coercion & Truthy/Falsy

List all 8 falsy values in JS. How do string concatenation and numeric coercion work under the hood?

7. Null vs Undefined & `typeof`

Compare intentional absence vs uninitialized state. Why does `typeof null` return `"object"`?

8. Pass by Value vs Pass by Reference

Demonstrate how argument passing in functions passes value references for mutable data types.

1. CORE FUNDAMENTALS (CONT.)

Continued

9. Shallow vs Deep Copying

Compare `Object.assign()` and spread syntax (`...`) with structured cloning and `JSON.parse(JSON.stringify())`.

10. Destructuring & Rest/Spread Syntax

Extract deeply nested properties and gather remaining parameters using array and object destructuring.

11. Optional Chaining & Nullish Coalescing

Differentiate `??` (null/undefined) from `||` (falsy values) alongside `?.` safety guards.

12. Strict Mode Mechanics

What changes occur when enabling `"use strict"`? (e.g., accidental global variables, duplicate parameter names).

2. FUNCTIONS – CORE MECHANISMS

~20 Questions

13. Declarations vs Expressions

How do function declarations hoist differently from anonymous/named function expressions assigned to variables?

14. Arrow Functions vs Normal Functions

Detail lexical `this`, missing `arguments` object, lack of `prototype`, and constructor limitations.

15. Explicit Context: `call()`, `apply()`, `bind()`

How do you manually set execution context? Contrast immediate invocation vs returning a bound function instance.

16. Closures & Lexical Environments

How does an inner function retain access to its outer scope variables after parent scope execution terminates?

17. Higher-Order & Callback Functions

Define functions accepting/returning other functions. Discuss asynchronous callback patterns and error handling.

18. IIFE (Immediately Invoked Function Expressions)

Purpose of IIFE pattern for private scoping before modern ES6 modules. Syntax nuances with trailing parenthetical calls.

19. Currying vs Partial Application

Transform functions taking multi-arguments into sequences of single-argument functions versus fixing N arguments.

20. Default Parameters & Arguments Object

Interaction between default argument values, missing inputs, and the array-like `arguments` structure.

3. SCOPE & HOISTING (TRICKY SCENARIOS)

~15 Hard Questions

21. Function Declaration Overriding Variable Declarations

Predict resolution order when variable and function share identical identifiers in a scope:

```
console.log(x); var x = 10; function x() {} // Outputs function definition
```

22. Variable Shadowing & Parameter Scope

Analyze function parameter initialization vs local variable re-declarations:

```
function test(a) { var a = 20; console.log(a); } test(10); // Outputs 20
```

23. Function Expression Hoisting Failure

What happens when invoking a hoisted variable declaration before function assignment?

```
foo(); // TypeError: foo is not a function  
var foo = function() {};
```

24. Temporal Dead Zone Boundary Edge Case

Identify the scope boundary where the TDZ triggers ReferenceError versus outer scope fallback.

4. OBJECTS & PROTOTYPES

~15 Questions

25. Object Creation Methods

Compare literals, `Object.create()`, constructor functions, and ES6 class syntax.

26. Computed Property Keys & Destructuring

Dynamically define object properties using bracket expressions and dynamic key extractors.

27. Object Immutability: Freeze vs Seal

Compare `Object.freeze()` (shallow static object) vs `Object.seal()` (prevent addition/deletion, allow modification).

28. Prototype Chain & Inheritance

Explain `__proto__` vs `prototype`, lookup traversal to `Object.prototype`, and `hasOwnProperty()` checks.

29. Static Object Utilities

Usage of `Object.keys()`, `Object.values()`, and `Object.entries()` in object transformations.

5. ARRAYS & HIGHER-ORDER METHODS

~15 Questions

30. Mutating vs Non-Mutating Array Methods

Categorize methods like `splice`, `sort` (mutating) vs `slice`, `concat`, `map`, `filter` (pure).

31. Deep Dive: `reduce()` & Method Chaining

Accumulator patterns: flattening, grouping objects, pipeline transformations, and initial value edge cases.

32. Search Mechanics: `find`, `findIndex`, `includes`, `some`, `every`

Short-circuit evaluation mechanisms across predicates and equality checks.

33. Array Sorting Gotchas

Why `[10, 2, 5].sort()` returns `[10, 2, 5]`. Custom comparator callbacks for numbers and strings.

6. ASYNCHRONOUS JAVASCRIPT & PROMISES

~25 Questions

34. Promise States & Lifecycle

Transitions from `pending` to `fulfilled` or `rejected`. Immutable settling behavior.

35. Promise Combinators

Compare `Promise.all()`, `Promise.allSettled()`, `Promise.race()`, and `Promise.any()` error scenarios.

36. Async/Await Syntactic Sugar

Underlying mechanics of `async/await` via Generators/Promises. Try/catch block positioning and uncaught rejections.

37. Sequential vs Parallel Async Execution

Difference between sequential `await` calls in loops vs `Promise.all(array.map(asyncFn))` concurrency.

38. Microtask Queue vs Macrotask Queue

How Promises (microtasks) take priority over `setTimeout` / `setInterval` (macrotasks) after call stack empties.

7. DOM, BROWSER APIS & OPTIMIZATION

~15 Questions

39. Event Propagation: Bubbling & Capturing

How events traverse down the DOM tree and bubble up. Role of `e.stopPropagation()` vs `e.preventDefault()`.

40. Event Delegation Pattern

Attaching a single listener to a parent element using `e.target` to handle dynamic child elements efficiently.

41. Browser Storage Options

Compare `localStorage`, `sessionStorage`, and `Cookies` (capacity, expiry, server headers).

42. Performance Techniques: Debounce vs Throttle

Implement delay timer resets vs fixed interval executions for scroll/resize/search inputs.

8. MODERN JAVASCRIPT & ES6+ FEATURES

~15 Questions

43. Iterators & Generator Functions

The Iterator protocol, `Symbol.iterator`, custom `yield` expressions, and lazy evaluation streams.

44. Keyed Collections: Map, Set, WeakMap, WeakSet

Garbage collection differences with weak reference collections vs standard Map/Set. Key type flexibility.

45. ES6 Classes & Private Fields

Syntactic wrapper around prototypes, `super()` binding, and native `#privateField` access boundaries.

9. ADVANCED JS CONCEPTS & ENGINE MECHANICS

~15 Questions

46. Execution Context Phases

Detail Creation Phase (Variable Environment setup, Scope Chain creation) vs Execution Phase (line-by-line assignment).

47. Garbage Collection & Memory Leaks

Mark-and-Sweep algorithm. Common leaks: unintended globals, uncleared timers, retained DOM references, closed variables.

48. Implement Custom Debounce & Throttle

```
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}
```

49. Custom Array Prototypes: map() and reduce()

```
Array.prototype.myMap = function(cb) {
  const result = [];
  for (let i = 0; i < this.length; i++) {
    result.push(cb(this[i], i, this));
  }
  return result;
};
```

50. Deep Array Flattening & Object Grouping

Write recursive array flattener without using native `Array.prototype.flat()`. Implement `groupBy(arr, key)`.

11. OUTPUT-BASED INTERVIEW CHALLENGES

51. Event Loop Task Scheduling Output

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");
// Output: A -> D -> C -> B
// Reason: Microtask queue (Promises) empties before Macrotask queue (Timers)
```

52. Scoped Loop Callbacks with Var vs Let

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // Outputs: 3, 3, 3
}
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log(j), 100); // Outputs: 0, 1, 2
}
```

53. Dynamic Binding vs Arrow Method Invocation

```
const obj = {
  name: "JS",
  regularFn: function() { return this.name; },
  arrowFn: () => this.name
};
// regularFn call returns "JS", arrowFn call returns undefined or global name
```